Like and Comment Functionality Implementation

Overview

This implementation adds interactive like and comment features to community posts in The Writer's Corner application, allowing authenticated users to engage with shared writing exercises.

Changes Made

1. Database Schema Updates (`app/prisma/schema.prisma`)

New Models Added:

PostLike Model:
- Tracks user likes on community posts (exercise submissions)
- Unique constraint ensures users can only like a post once
- Cascade delete when user or post is deleted
- Indexed on `postId` for performance

PostComment Model:
- Stores user comments on community posts
- Includes `createdAt` and `updatedAt` timestamps
- Cascade delete when user or post is deleted
- Indexed on `postId` for performance

User Model Updates:
- Added `postLikes` relation
- Added `postComments` relation

CommunityPost Model Updates:
- Added `likes` relation
- Added `comments` relation

2. API Routes Created

Like Functionality (`app/app/api/community/posts/[postId]/like/route.ts`)

- **POST**: Like a post
- Validates user authentication
- Checks if post exists
- Prevents duplicate likes
- Returns updated like count and liked status
- **DELETE**: Unlike a post
- Validates user authentication
- Removes user's like
- Returns updated like count and liked status

Comment Functionality (`app/app/api/community/posts/[postId]/comments/route.ts`)

- **GET**: Fetch all comments for a post
- Returns comments with user information
- Ordered by creation date (oldest first)
- **POST**: Create a new comment
- Validates user authentication
- Validates comment content
- Returns created comment with user information

Comment Management (`app/app/api/community/comments/[commentId]/route.ts`)

- **PATCH**: Edit a comment
- Validates user is the comment author
- Updates comment content and timestamp
- Returns updated comment
- **DELETE**: Delete a comment
- Validates user is the comment author
- Removes comment from database
- Returns success status

3. Community Posts API Updates (`app/app/api/community/posts/route.ts`)

Enhanced the GET endpoint to include:
- Like counts for each post (aggregated)
- Comment counts for each post (aggregated)
- User's like status for each post (`isLikedByUser`)

4. Frontend Component Updates (`app/components/community/community-overview.tsx`)

New State Management:

- `expandedComments`: Tracks which posts have comments expanded
- `comments`: Stores fetched comments by post ID
- `newComment`: Manages new comment input per post
- `editingComment`: Tracks which comment is being edited
- `editContent`: Stores edited comment content
- `loadingComments`: Tracks loading state for comments

New Functions:

- `handleLike()`: Toggle like/unlike on a post
- `fetchComments()`: Load comments for a post
- `toggleComments()`: Expand/collapse comment section
- `handleAddComment()`: Submit a new comment
- `handleEditComment()`: Update an existing comment
- `handleDeleteComment()`: Remove a comment
- `getUserDisplayName()`: Format user display names consistently

UI Enhancements:

- **Like Button**:
- Shows like count or “Like” text
- Visual indication when user has liked (filled heart icon, rust color)
- Optimistic UI updates
- **Comment Button**:
- Shows comment count or “Comment” text
- Toggles comment section visibility
- **Comment Section** (expandable):
- Comment input textarea with “Post Comment” button
- List of existing comments with author and timestamp
- Edit/delete buttons for user’s own comments (icon buttons)
- “edited” indicator for modified comments
- Loading state while fetching comments
- Empty state message when no comments exist

New Imports:

- `useSession` from next-auth/react for user authentication
- `Edit2`, `Trash2`, `Send` icons from lucide-react
- `Textarea` component for comment input

Features

Like Functionality

✅ Authenticated users can like/unlike posts
✅ Visual feedback when post is liked (filled heart, color change)
✅ Like count displayed on each post
✅ Prevents duplicate likes (database constraint)
✅ Real-time UI updates

Comment Functionality

✅ Authenticated users can comment on posts
✅ Comment count displayed on each post
✅ Expandable comment section
✅ Comments show author name and timestamp
✅ Users can edit their own comments
✅ Users can delete their own comments
✅ “Edited” indicator for modified comments
✅ Loading states for better UX
✅ Empty state messaging

Security & Validation

- All endpoints require authentication via NextAuth session
- Comment edit/delete operations verify user ownership

- Input validation for comment content (non-empty, trimmed)
- Proper error handling with appropriate HTTP status codes
- Database constraints prevent data integrity issues

Database Migration

A migration file has been created at `app/prisma/migrations/add_likes_and_comments.sql` documenting the schema changes.

To apply the schema changes:

```
cd app
npx prisma migrate dev --name add_likes_and_comments
npx prisma generate
```

Design Decisions

1. **Post Reference**: Uses `ExerciseSubmission` as the post entity since community posts are public exercise submissions
2. **Comment Ordering**: Comments display oldest-first to maintain conversation flow
3. **Edit/Delete Permissions**: Only comment authors can modify their comments
4. **UI/UX**: Follows existing vintage/typewriter aesthetic with sepia tones and serif fonts
5. **Performance**: Aggregated counts fetched efficiently using Prisma's `groupBy`
6. **User Experience**: Optimistic UI updates for likes, loading states for comments

Testing Recommendations

1. Test like/unlike functionality with multiple users
2. Verify comment CRUD operations
3. Test permission checks (edit/delete own comments only)
4. Verify UI updates correctly after each action
5. Test with posts that have no likes/comments
6. Test concurrent operations (multiple users interacting simultaneously)

Future Enhancements (Optional)

- Nested replies to comments
- Comment reactions/likes
- Real-time updates using WebSockets
- Notification system for new comments
- Comment pagination for posts with many comments
- Rich text formatting in comments
- @mentions in comments